Week 3

1. AIM

To interface **HC-05 Bluetooth module** with **Arduino UNO** and control an LED using a smartphone by sending `'1'` (ON) and `'0'` (OFF).

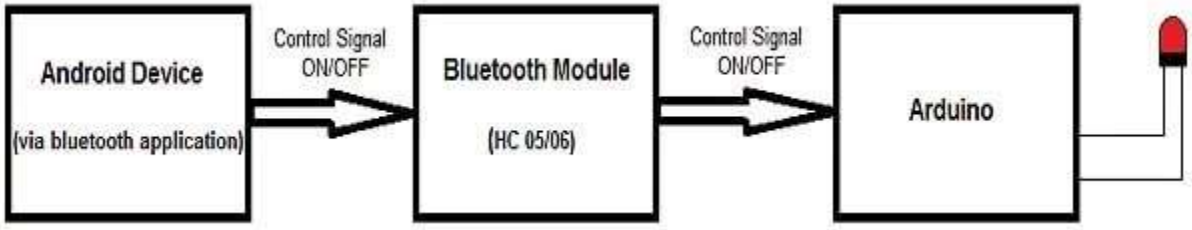
2. HARDWARE REQUIREMENTS

- Arduino UNO
- HC-05 Bluetooth Module
- LED (or onboard LED on pin 13)
- Connecting wires
- Android smartphone with Bluetooth
- USB cable for programming and powering Arduino

3. PROCEDURE

1. Connect HC-05 module to Arduino:
    - VCC → 5V
    - GND → GND
    - TX → Pin 10 (RX)
    - RX → Pin 9 (TX)
2. Connect LED:
    - Positive → Pin 13
    - Negative → GND through resistor
3. Upload the program to Arduino using USB cable.
4. Pair the smartphone with HC-05 (default password: 1234 or 0000).
5. Open a Bluetooth terminal/app and connect to HC-05.
6. Send `'1'` to turn ON the LED and `'0'` to turn OFF the LED.
7. Observe LED behavior and Serial Monitor output.

(There are three main parts to this project. An Android smart phone, a Bluetooth transceiver, and an Arduino.

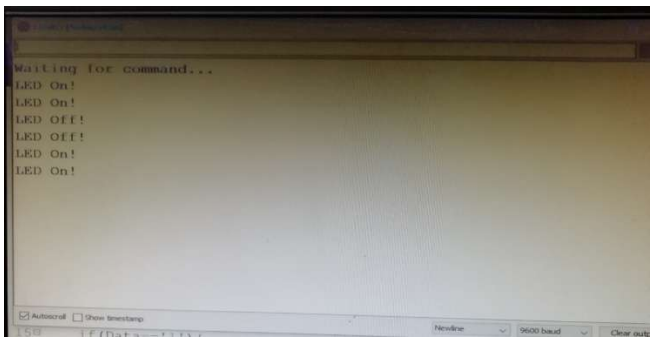


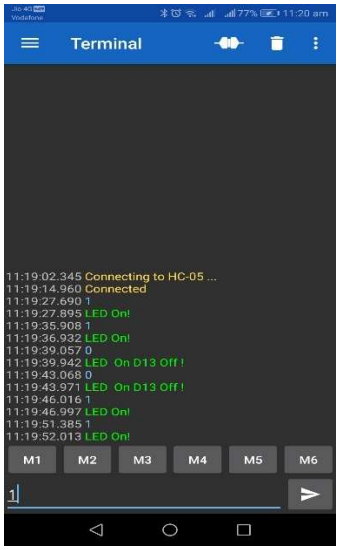
HC 05/06 works on serial communication.)→ref

4. SOURCE CODE

```
#include <SoftwareSerial.h>

SoftwareSerial BT(10, 9); // RX, TX
int LED = 13;
char data;

void setup() {
  pinMode(LED, OUTPUT);

  Serial.begin(9600);
  BT.begin(9600);

  Serial.println("System Ready");
  BT.println("Send 1 for ON, 0 for OFF");
}

void loop() {
  if (BT.available()) {
    data = BT.read();

    Serial.print("Received: ");
    Serial.println(data);

    if (data == '1') {
      digitalWrite(LED, HIGH);
      Serial.println("LED ON");
      BT.println("LED ON");
    }
    else if (data == '0') {
      digitalWrite(LED, LOW);
      Serial.println("LED OFF");
      BT.println("LED OFF");
    }
  }
  delay(100);
}
```

5. EXPECTED OUTPUT





- When `'1'` is sent from the smartphone, the LED turns **ON**.
- When `'0'` is sent, the LED turns **OFF**.
- Serial Monitor displays:
  - "Received: 1 → LED ON"
  - "Received: 0 → LED OFF"

Week 4

1. AIM

To implement **RFID using Arduino UNO** for **writing and reading data** from an RFID tag using RC522 module.

2. HARDWARE REQUIREMENTS

- Arduino UNO
- RC522 RFID Module
- RFID Tag / Card (MIFARE type)
- Connecting wires
- USB cable

3. PROCEDURE

◇ Understanding Equipment

- **RFID RC522 Module (Reader)**
  Acts as a transmitter and receiver. It generates a **13.56 MHz electromagnetic field** to detect RFID tags.
- **RFID Tag/Card (Passive Device)**
  Contains a **microchip + antenna**. It has no battery and gets powered when placed near the reader.
- **Working Principle:**
  When the tag comes near the reader:
  - Reader creates electromagnetic field
  - Tag gets powered
  - Tag sends stored data (UID or written data) back to reader
  - Arduino processes and displays it

◇ Connections (Verified for Arduino UNO)

RC522 Pin	Arduino UNO Pin
VCC	3.3V ⚠

RC522 Pin	Arduino UNO Pin
GND	GND
RST	Pin 9
SDA (SS)	Pin 10
MOSI	Pin 11
MISO	Pin 12
SCK	Pin 13

Uses SPI Communication

◇ Library Installation

1. Open Arduino IDE
2. Go to **Sketch → Include Library → Manage Libraries**
3. Search **MFRC522**
4. Install the library

◇ Writing Data to RFID Tag

1. Open example code for writing (provided program)
2. Upload code to Arduino
3. Open Serial Monitor
4. Enter data (name) ending with #
5. Place RFID tag near reader

👉 Data is written into tag memory

◇ Reading Data from RFID Tag

1. Upload reading program (given code)
2. Open Serial Monitor
3. Place RFID tag near reader

👉 Stored data (name/UID) will be displayed

◇ Verification

- First write data
- Then read using reading program
- Confirm stored data is correctly retrieved

4. SOURCE CODE

◇ Writing Program

(Uses MFRC522 library to store data in RFID tag)

◇ Reading Program

(Reads stored data from RFID tag)

👉 Already provided and used directly from example

5. EXPECTED OUTPUT

◇ Writing:

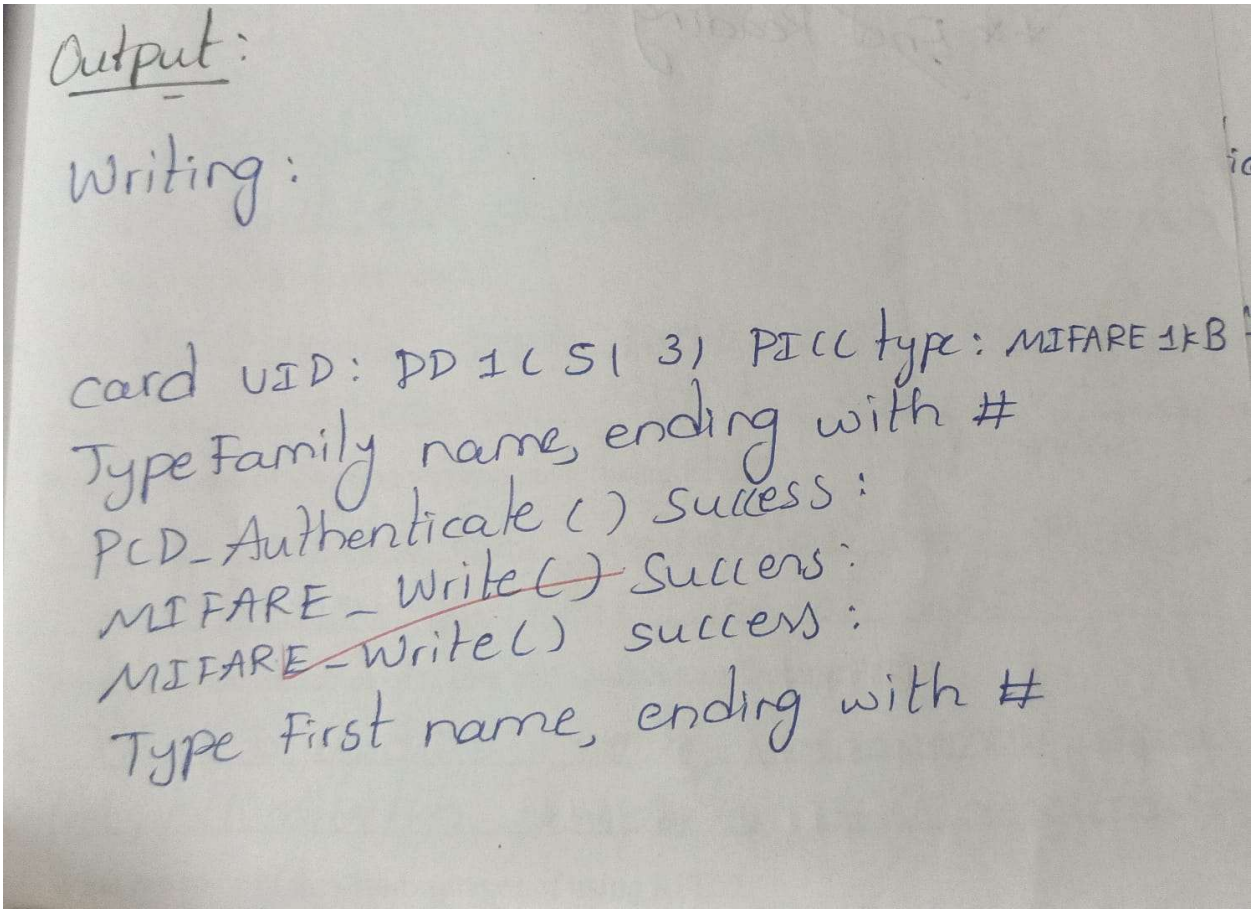
- Serial Monitor asks for input (name)
- Displays:
  - "Write success"

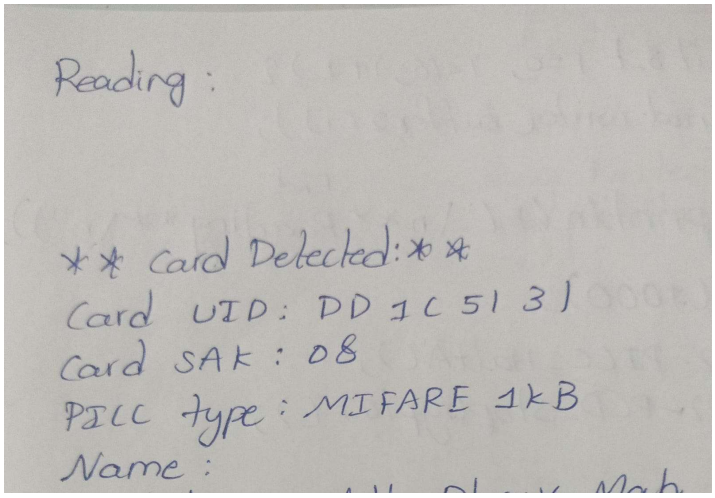
◇ Reading:

- When RFID tag is scanned:
  - UID is displayed
  - Stored data (name) is printed

Output:

Writing:





Week 5

1. AIM

To monitor **temperature and humidity** using **DHT11/DHT22 sensor** with Arduino UNO.

2. HARDWARE REQUIREMENTS

- Arduino UNO
- DHT11 or DHT22 Temperature and Humidity Sensor
- Connecting wires
- Breadboard
- USB cable

3. PROCEDURE

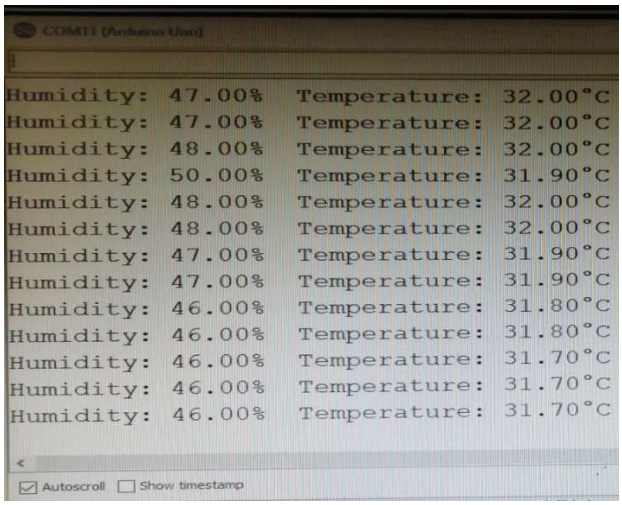
1. **Understand the sensor:**
   - DHT11/DHT22 are digital sensors used to measure **temperature and humidity**.
   - DHT22 provides **higher accuracy and wider range** compared to DHT11.
2. **Make connections:**
   - Pin 1 (VCC) → 5V
   - Pin 2 (Data) → Pin 2 (Arduino)
   - Pin 4 (GND) → GND
3. **Install library:**
   - Open Arduino IDE
   - Go to **Sketch → Include Library → Manage Libraries**
   - Install **DHT sensor library**
4. **Upload code:**
   - Select correct sensor type in code:
     - `DHT11` or `DHT22`
   - Upload program to Arduino
5. **Observe output:**
   - Open Serial Monitor
   - Temperature and humidity values will be displayed continuously

4. SOURCE CODE

```
#include "DHT.h"

#define DHTPIN 2
#define DHTTYPE DHT11   // change to DHT22 if using DHT22

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(9600);
  dht.begin();
}

void loop() {
  delay(2000);

  float h = dht.readHumidity();
  float t = dht.readTemperature();

  if (isnan(h) || isnan(t)) {
    Serial.println("Sensor Error");
    return;
  }

  Serial.print("Humidity: ");
  Serial.print(h);
  Serial.print("%  Temp: ");
  Serial.print(t);
  Serial.println(" C");
}
```

5. EXPECTED OUTPUT

```
Humidity: 65%  Temp: 27 C
```



Week 6

1. AIM

To monitor sensor data using **NodeMCU (ESP8266)** and upload it to **ThingSpeak IoT Cloud**.

2. HARDWARE REQUIREMENTS

- NodeMCU (ESP8266)
- IR Sensor
- Connecting wires
- WiFi connection
- USB cable

3. PROCEDURE

1. **Understand the components:**
    - **NodeMCU (ESP8266):** Microcontroller with built-in WiFi used for IoT communication
    - **IR Sensor:** Detects object presence and gives digital output (0 or 1)
    - **ThingSpeak:** Cloud platform used to store and visualize sensor data
2. **Make connections:**
    - IR Sensor VCC → 3.3V
    - IR Sensor GND → GND
    - IR Sensor OUT → D2
3. **Setup ThingSpeak:**
    - Create a channel
    - Enable Field1
    - Copy the **Write API Key**
4. **Program configuration:**
    - Enter WiFi SSID and password
    - Enter ThingSpeak API key
5. **Upload the program to NodeMCU**
6. **Working:**
    - NodeMCU connects to WiFi
    - Reads IR sensor value
    - Sends data to ThingSpeak using HTTP
7. **Observation:**
    - Serial Monitor shows sensor status
    - ThingSpeak displays graph of uploaded data

4. SOURCE CODE

```
#include <ESP8266WiFi.h>

const char* ssid     = "xxx";
const char* pass     = "xxx";
const char* apiKey   = "xxx";
const char* server   = "api.thingspeak.com";

#define IR_PIN D2

WiFiClient client;

void setup() {
  Serial.begin(115200);
  pinMode(IR_PIN, INPUT);

  WiFi.begin(ssid, pass);
  Serial.print("Connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }
  Serial.println("\nWiFi Connected!");
}

void loop() {
  int value = digitalRead(IR_PIN);

  Serial.println(value == 0 ? "Object detected" : "No object");

  if (client.connect(server, 80)) {
    String data = "api_key=" + String(apiKey) + "&field1=" + String(value);

    client.print("POST /update HTTP/1.1\r\n");
    client.print("Host: api.thingspeak.com\r\n");
    client.print("Content-Type: application/x-www-form-urlencoded\r\n");
    client.print("Content-Length: " + String(data.length()) + "\r\n\r\n");
    client.print(data);
    client.stop();

    Serial.println("Data sent!");
  } else {
    Serial.println("Connection failed.");
  }

  delay(15000); // ThingSpeak limit
}
```

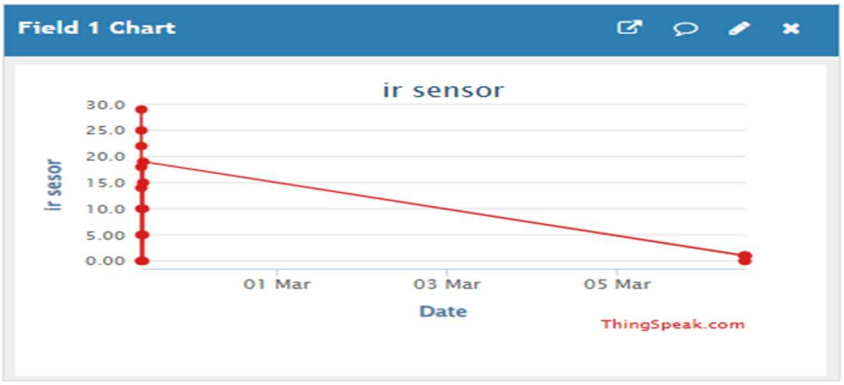
5. EXPECTED OUTPUT

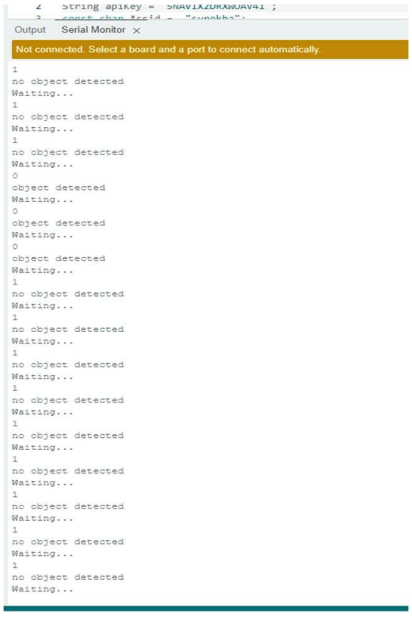
◇ Serial Monitor:

```
Object detected
Data sent!

No object
Data sent!
```

◇ ThingSpeak Cloud:

- Data uploaded to **Field1**
- Graph shows:
  - `0 → Object detected`
  - `1 → No object`





Week 7

1. AIM

To upload **temperature and humidity data** to **ThingSpeak cloud** using **NodeMCU (ESP8266) and DHT11 sensor**.

2. HARDWARE REQUIREMENTS

- NodeMCU (ESP8266)
- DHT11 Temperature and Humidity Sensor
- Connecting wires
- Breadboard
- WiFi network

3. PROCEDURE

1. **Understand components:**
   - **NodeMCU (ESP8266):** Used for WiFi-based IoT communication
   - **DHT11 Sensor:** Measures temperature and humidity
   - **ThingSpeak:** Cloud platform to store and visualize data
2. **Make connections:**
   - DHT11 VCC → 3.3V
   - DHT11 GND → GND
   - DHT11 Data → D3
3. **Setup ThingSpeak:**
   - Create a channel
   - Enable Field1 (Temperature) and Field2 (Humidity)
   - Copy the **Write API Key**
4. **Program setup:**
   - Enter WiFi SSID and password
   - Enter ThingSpeak API key
5. **Upload code to NodeMCU**
6. **Working:**
   - NodeMCU connects to WiFi
   - Reads temperature and humidity from DHT11
   - Sends data to ThingSpeak cloud
7. **Observation:**
   - Serial Monitor shows values
   - ThingSpeak displays graphs for temperature and humidity

4. SOURCE CODE

```
#include <ESP8266WiFi.h>
#include "DHT.h"

#define DHTPIN D3
#define DHTTYPE DHT11

const char* ssid = "xxx";
const char* pass = "xxx";
const char* server = "api.thingspeak.com";
String apiKey = "xxx";

DHT dht(DHTPIN, DHTTYPE);
WiFiClient client;

void setup() {
  Serial.begin(115200);
  dht.begin();

  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }
  Serial.println("WiFi Connected");
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {
    Serial.println("Sensor Error");
    return;
  }

  Serial.print("Temp: ");
  Serial.print(t);
  Serial.print(" C  Hum: ");
  Serial.println(h);

  if (client.connect(server, 80)) {
    String data = "api_key=" + apiKey + "&field1=" + String(t) + "&field2=" +
String(h);

    client.print("POST /update HTTP/1.1\r\n");
    client.print("Host: api.thingspeak.com\r\n");
    client.print("Content-Type: application/x-www-form-urlencoded\r\n");
    client.print("Content-Length: " + String(data.length()) + "\r\n\r\n");
    client.print(data);

    client.stop();
    Serial.println("Data sent");
  } else {
    Serial.println("Connection failed");
  }

  delay(15000); // ThingSpeak limit}
```

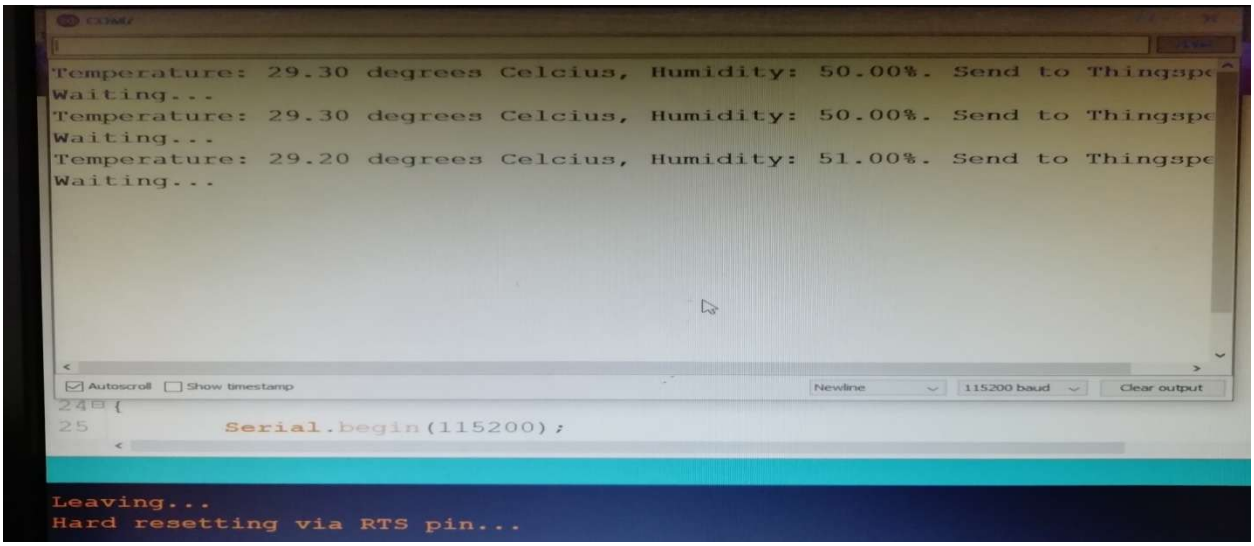
5. EXPECTED OUTPUT

- Serial Monitor:

```
Temp: 27 C  Hum: 65
Data sent
```

- ThingSpeak Cloud:

- **Field1 → Temperature**
- **Field2 → Humidity**
- Graph shows real-time variation of values



Week 8

1. AIM

To retrieve **temperature and humidity data** from **ThingSpeak cloud** using **NodeMCU (ESP8266)**.

2. HARDWARE REQUIREMENTS

- NodeMCU (ESP8266)
- Connecting wires (optional)
- Breadboard (optional)
- WiFi network

3. PROCEDURE

1. **Understand components:**
    - **NodeMCU (ESP8266):** Used to connect to internet and retrieve cloud data
    - **ThingSpeak:** Stores temperature and humidity data in cloud fields
2. **Setup ThingSpeak:**
    - Create a channel
    - Ensure:
        - Field1 → Temperature
        - Field2 → Humidity
    - Copy **Channel ID** and **Read API Key**
3. **Program setup:**
    - Enter WiFi SSID and password
    - Enter Channel ID and Read API Key
4. **Upload program to NodeMCU**
5. **Working:**
    - NodeMCU connects to WiFi
    - Reads temperature and humidity data from ThingSpeak
    - Displays values on Serial Monitor
6. **Observation:**
    - Serial Monitor shows temperature and humidity values retrieved from cloud

4. SOURCE CODE (SIMPLIFIED & CLEAN)

```
#include <ESP8266WiFi.h>
#include "ThingSpeak.h"

const char* ssid = "xxx";
const char* pass = "xxx";

unsigned long channelID = 123456;      // your channel ID
const char* readAPI = "xxx";           // read API key
```

```
WiFiClient client;

void setup() {
  Serial.begin(115200);

  WiFi.begin(ssid, pass);
  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }
  Serial.println("WiFi Connected");

  ThingSpeak.begin(client);
}

void loop() {
  long temp = ThingSpeak.readLongField(channelID, 1, readAPI);
  long hum  = ThingSpeak.readLongField(channelID, 2, readAPI);

  if (ThingSpeak.getLastReadStatus() == 200) {
    Serial.print("Temp: ");
    Serial.print(temp);
    Serial.print(" C  Hum: ");
    Serial.println(hum);
  } else {
    Serial.println("Error reading data");
  }

  delay(15000); // safe interval
}
```

5. EXPECTED OUTPUT

◇ Serial Monitor:

```
Temp: 27 C  Hum: 65
```

◇ Working Result:

- Data is **retrieved from ThingSpeak cloud**
- Temperature and humidity values are displayed continuously


```
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'CO
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
Temperature: 29
Humidity: 36
```

Week 9

1. AIM

To create a **TCP server using NodeMCU** and send **temperature and humidity data** to a TCP client.

2. HARDWARE REQUIREMENTS

- NodeMCU (ESP8266)
- DHT11 / DHT22 Temperature and Humidity Sensor
- Connecting wires
- Breadboard
- WiFi network
- Smartphone with TCP Client app

3. PROCEDURE

1. **Understand the components:**
    - **NodeMCU (ESP8266):** Acts as a **TCP server** using WiFi
    - **DHT Sensor:** Measures temperature and humidity
    - **TCP Client App:** Used to receive data from server

2. **Make connections:**
    - DHT VCC → 3.3V
    - DHT GND → GND
    - DHT Data → D3

3. **Program setup:**
    - Enter WiFi SSID and password in code
    - Upload the program to NodeMCU

4. **WiFi connection:**
    - NodeMCU connects to WiFi
    - IP address is displayed on Serial Monitor

5. **Start TCP server:**
    - Server starts on **port 8080**

6. **Connect using mobile:**
    - Open **TCP Client app**
    - Enter:
        - IP address (from Serial Monitor)
        - Port: 8080

- Connect to server

7. **Working:**
   - When client connects:
     - NodeMCU reads temperature and humidity
     - Sends data to mobile
     - Values are displayed

8. **Observation:**
   - Data appears on:
     - TCP client app
     - Serial Monitor

4. SOURCE CODE

```
#include <ESP8266WiFi.h>
#include "DHT.h"

const char* ssid = "xxx";
const char* password = "xxx";

WiFiServer server(8080);

#define DHTPIN D3
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.println("Connecting...");
  }

  Serial.print("IP: ");
  Serial.println(WiFi.localIP());

  server.begin();
  dht.begin();
}

void loop() {
  WiFiClient client = server.available();

  if (client) {
    Serial.println("Client Connected");
```

```
    while (client.connected()) {
      float t = dht.readTemperature();
      float h = dht.readHumidity();

      client.print("Temp: ");
      client.print(t);
      client.print(" C  Hum: ");
      client.println(h);

      Serial.print("Temp: ");
      Serial.print(t);
      Serial.print("  Hum: ");
      Serial.println(h);

      delay(2000);
    }

    client.stop();
    Serial.println("Client Disconnected");
  }
}
```

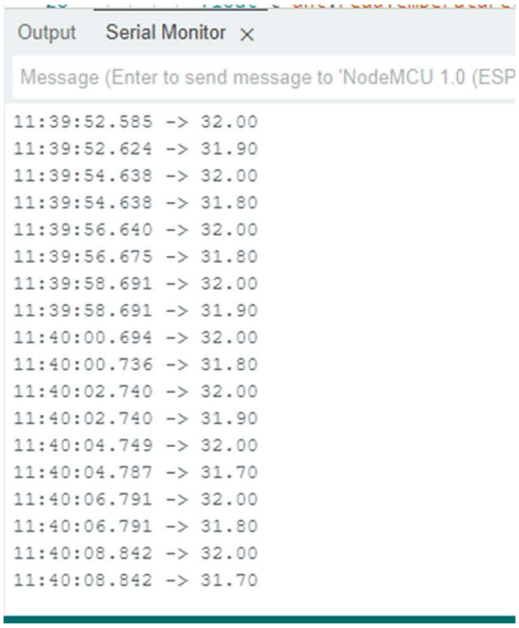
5. EXPECTED OUTPUT

- TCP Client App:

```
Temp: 27 C  Hum: 65
Temp: 27 C  Hum: 65
```

- Serial Monitor:

```
Client Connected
Temp: 27  Hum: 65
Client Disconnected
```



Week 10

1. AIM

To create a **UDP communication system using NodeMCU** and send **temperature and humidity data** to a UDP client.

2. HARDWARE REQUIREMENTS

- NodeMCU (ESP8266)
- DHT11 / DHT22 Sensor
- Connecting wires
- Breadboard
- WiFi network
- Smartphone/Laptop with UDP client app

3. PROCEDURE

1. **Understand components:**
    - **NodeMCU (ESP8266):** Sends data using UDP protocol
    - **DHT Sensor:** Measures temperature and humidity
    - **UDP Client App:** Receives data sent by NodeMCU

2. **Make connections:**
    - DHT VCC → 3.3V
    - DHT GND → GND
    - DHT Data → D3

3. **Program setup:**
    - Enter WiFi SSID and password
    - Enter **client IP address** and **port (1234)**

4. **Upload program to NodeMCU**

5. **WiFi connection:**
    - NodeMCU connects to WiFi

6. **Open UDP client app:**
    - Enter:
        - IP address (your device IP)
        - Port: 1234

7. **Working:**
    - NodeMCU reads temperature and humidity
    - Sends data continuously using UDP
    - UDP client receives and displays data

8. **Observation:**
    - Data visible on:
        - UDP client app
        - Serial Monitor

4. SOURCE CODE

```
#include <ESP8266WiFi.h>
#include <WiFiUdp.h>
#include "DHT.h"

const char* ssid = "xxx";
const char* password = "xxx";

const char* clientIP = "192.168.x.x"; // your phone/laptop IP
const int port = 1234;

#define DHTPIN D3
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
WiFiUDP udp;

void setup() {
  Serial.begin(115200);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(1000);
    Serial.print(".");
  }

  Serial.println("WiFi Connected");
  dht.begin();
}

void loop() {
  float t = dht.readTemperature();
  float h = dht.readHumidity();

  if (isnan(t) || isnan(h)) {
    Serial.println("Sensor Error");
    return;
  }

  udp.beginPacket(clientIP, port);
```

```
  udp.print("Temp: ");
  udp.print(t);
  udp.print(" C  Hum: ");
  udp.print(h);
  udp.endPacket();

  Serial.print("Sent -> Temp: ");
  Serial.print(t);
  Serial.print(" Hum: ");
  Serial.println(h);

  delay(2000);
}
```

5. EXPECTED OUTPUT

- UDP Client App:

```
Temp: 27 C  Hum: 65
Temp: 27 C  Hum: 65
```

- Serial Monitor:

```
Sent -> Temp: 27 Hum: 65
```